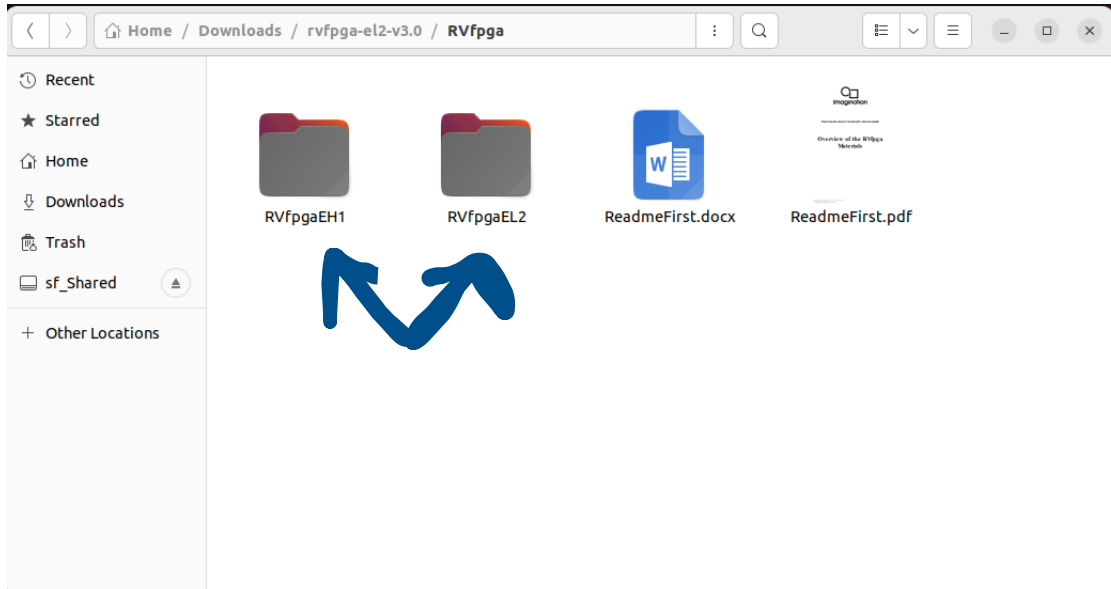
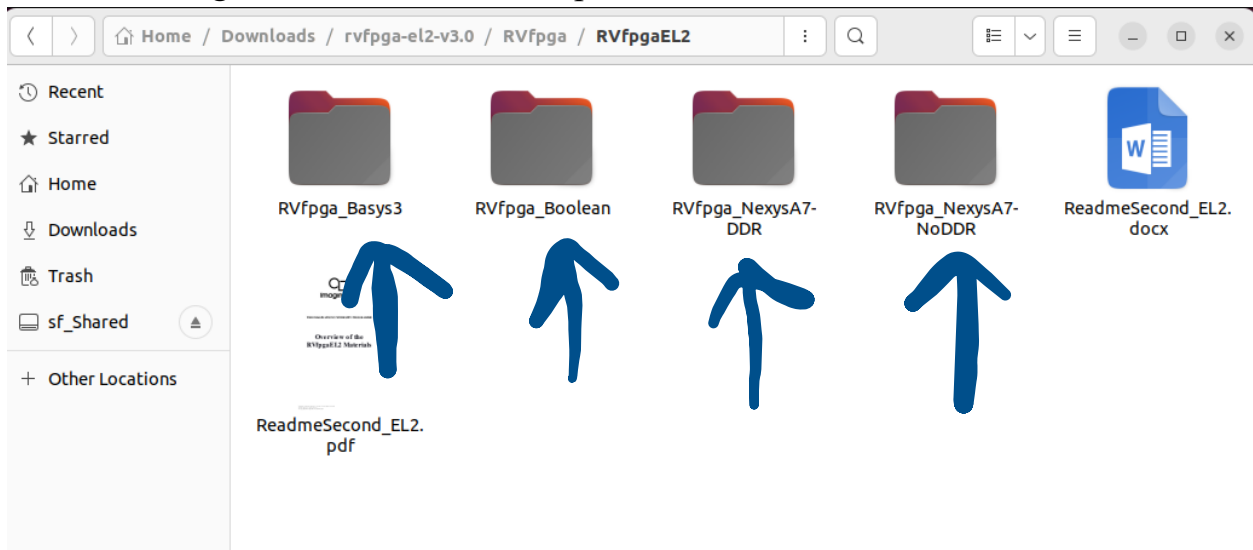


Documentation about running an RVFPGA project:

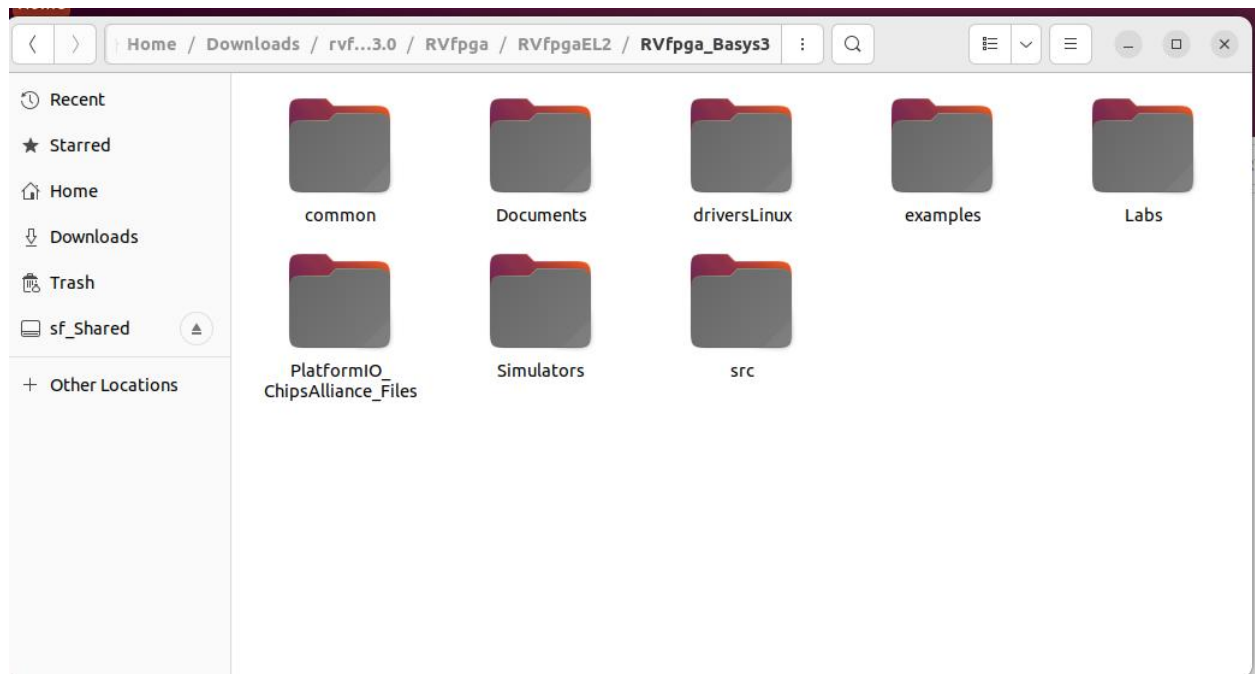
Firstly we'll explore how to run a predefined example from the course:



After choosing one of the cores and a platform



Every platform has the following structure: common , Documents, driversLinux, examples, Labs, PlatformIO_ChipsAlliance_Files, Simulators, src

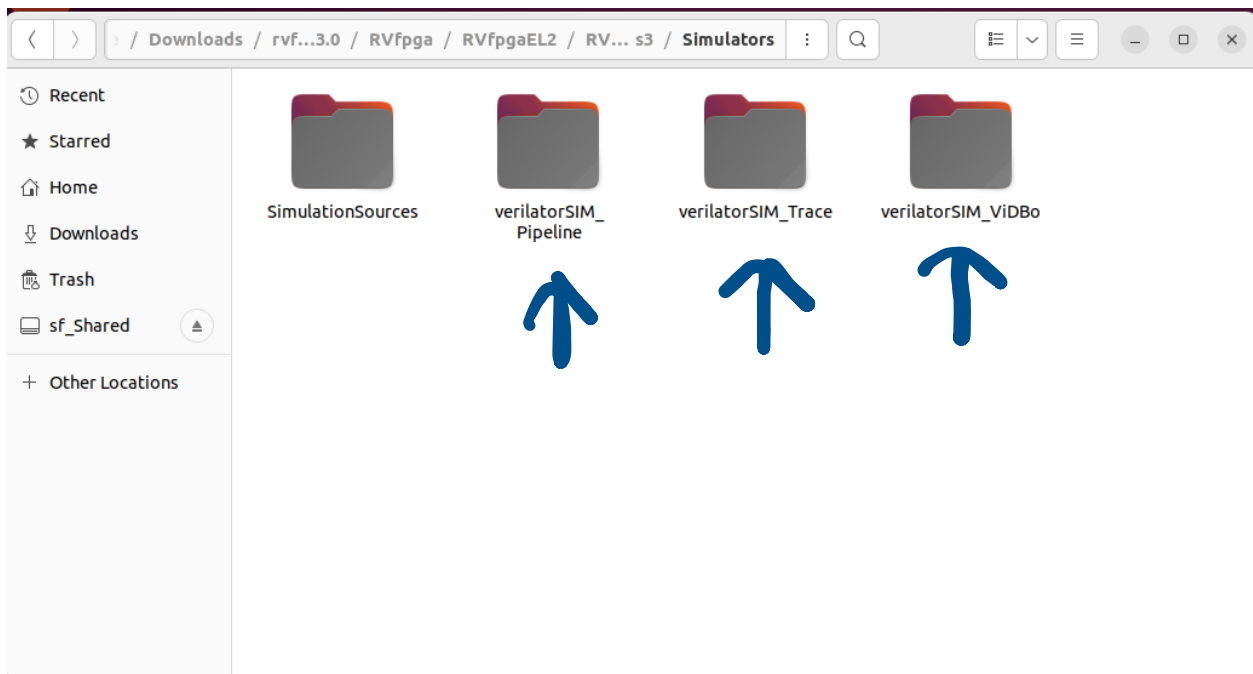


For this documentation we only care about “examples” and “Simulators” folders.

So before running anything we should have the simulators ready to run.

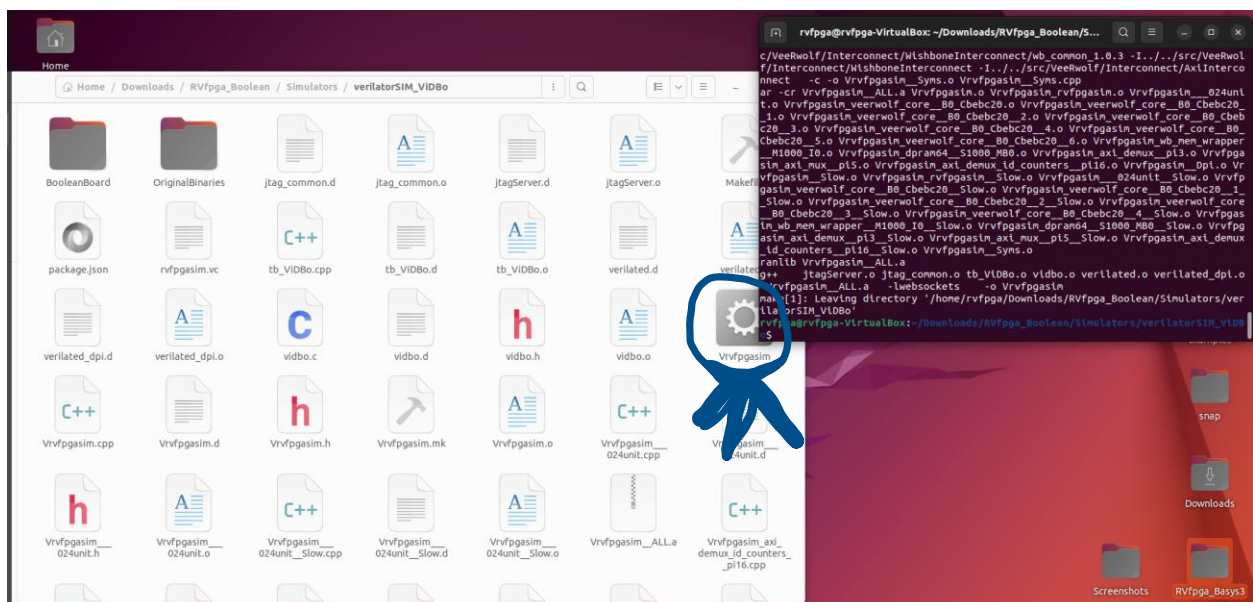
We have 4 simulators, from which 3 are based on verilator.

We have Pipeline, Trace, ViDBo, and Whisper simulator, the first three being based on verilator. The fourth one being integrated into catapult sdk.



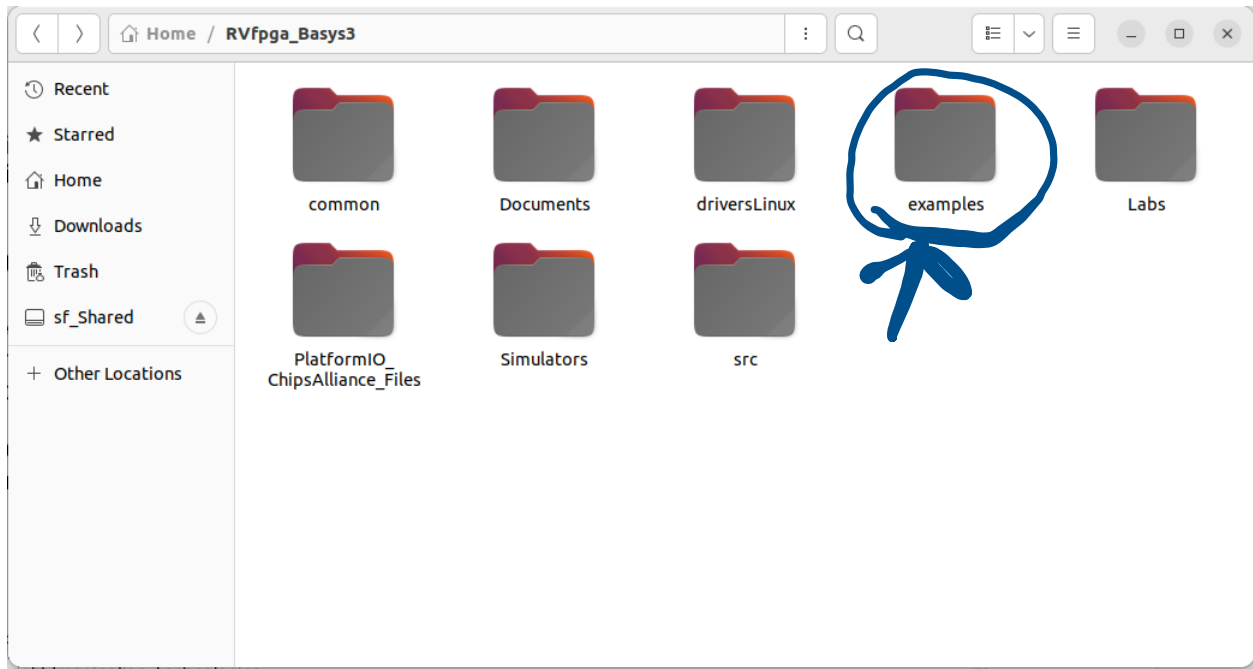
Assuming we'll be using all of the simulators at some point, and since we only need to create the simulators once, we can consecutively open a terminal in each of the folders and run the command “make clean” to clean up the old/bad files if there are any, then run “make”.

If everything worked fine, we should have the simulation files ready to run, no errors in the terminal and a “Vrvfpgasim” binary should be ready to execute.

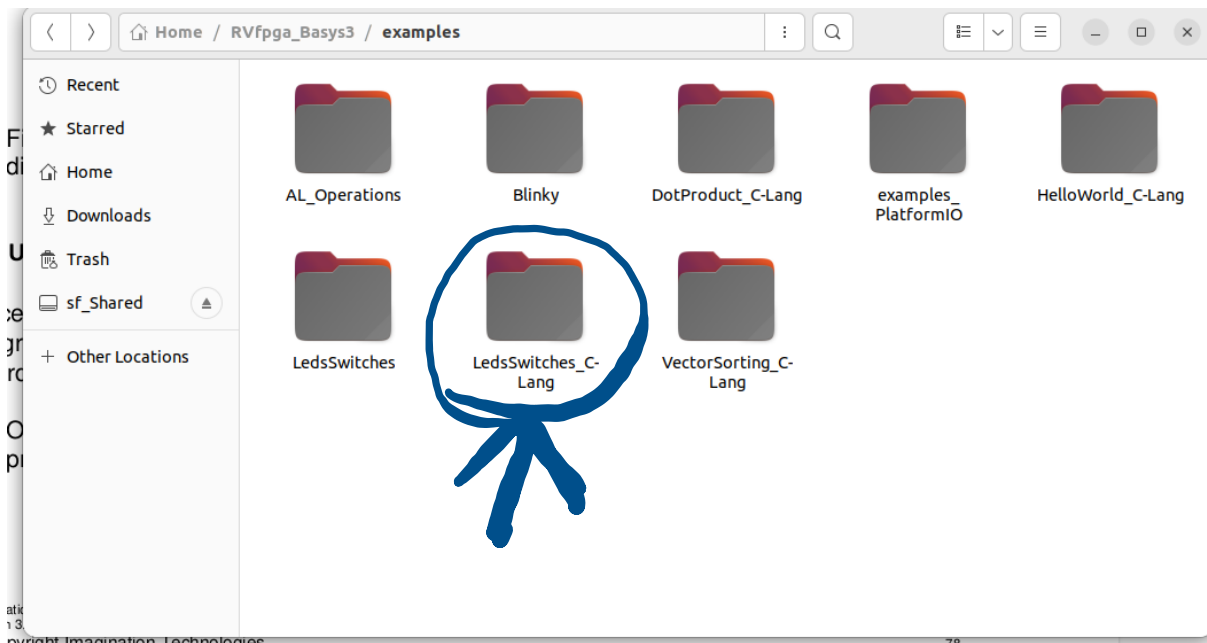


After doing this we're ready to proceed in executing examples.

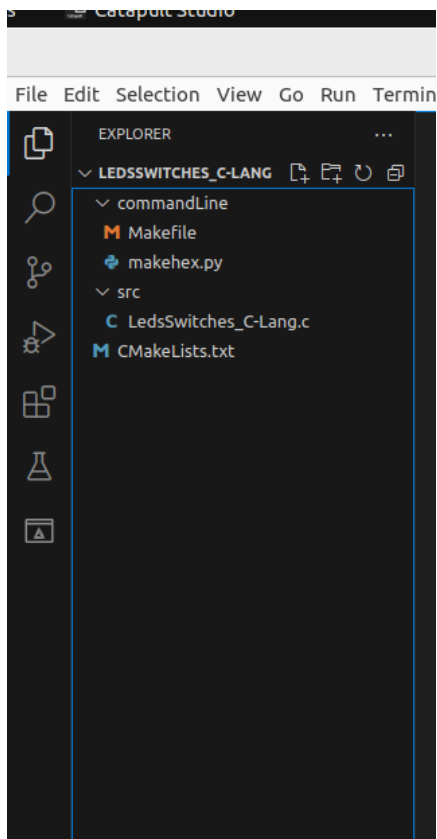
So we'll go back to the "examples" folder from the platform structure.



For the visual simulation (Vidbo) we'll choose the "LedsSwiches_C-lang" example.

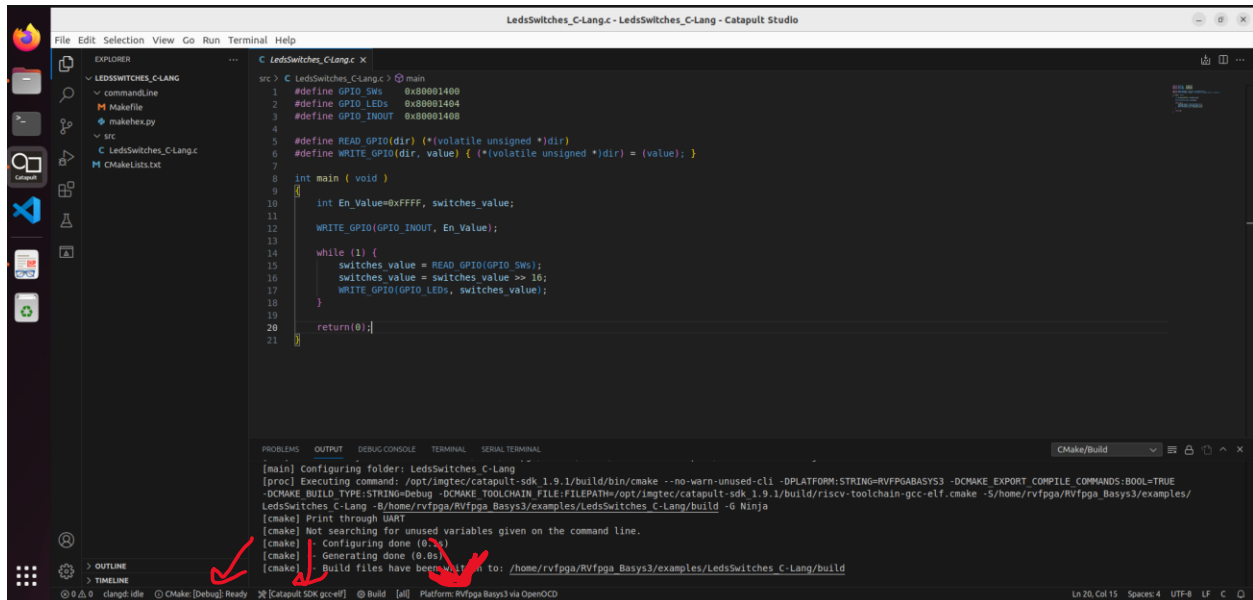


We'll open the LedsSwitches_C-Lang folder in Catapult SDK.



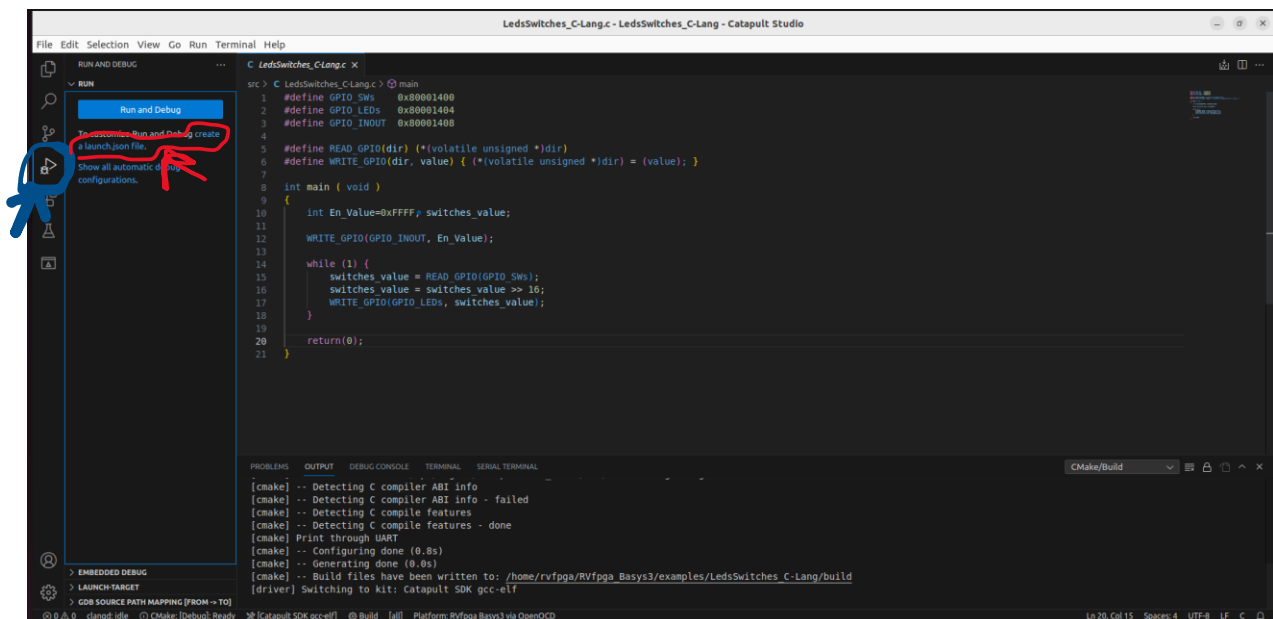
The folder structure would probably look something like this. Here, the first step we need to do is to choose our Cmake Debug state, the toolchain kit for the environment we'll run the code in, and the platform we want to run the code on.

We can find all of these settings in the bottom panel of the ide

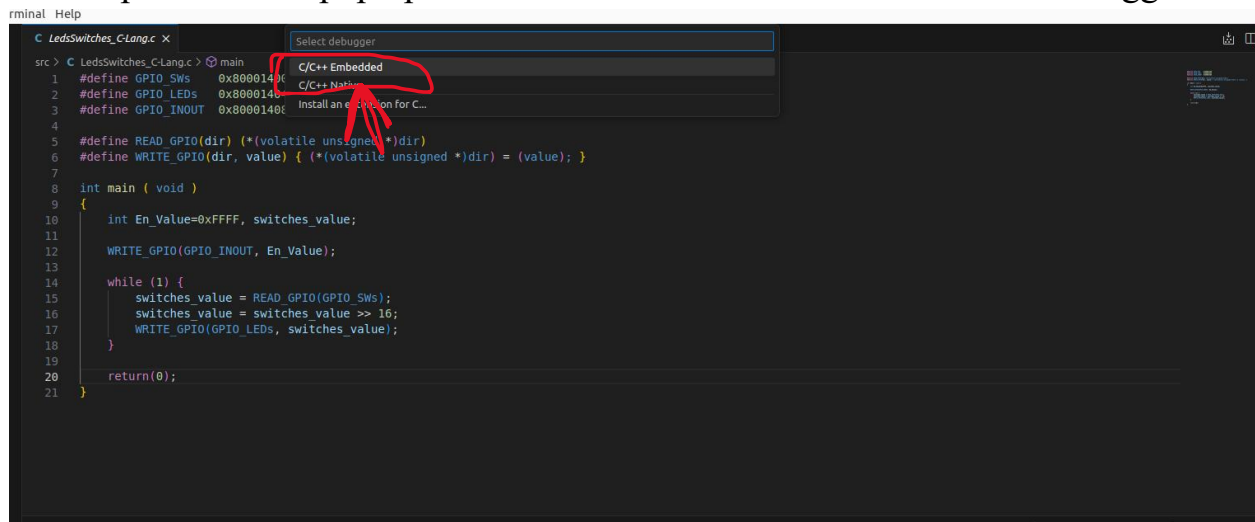


In any case we choose debug, the kit should be “Catapult Sdk gcc-elf” and the platform is the board that you chose, in my case Basys3 via OpenOCD.

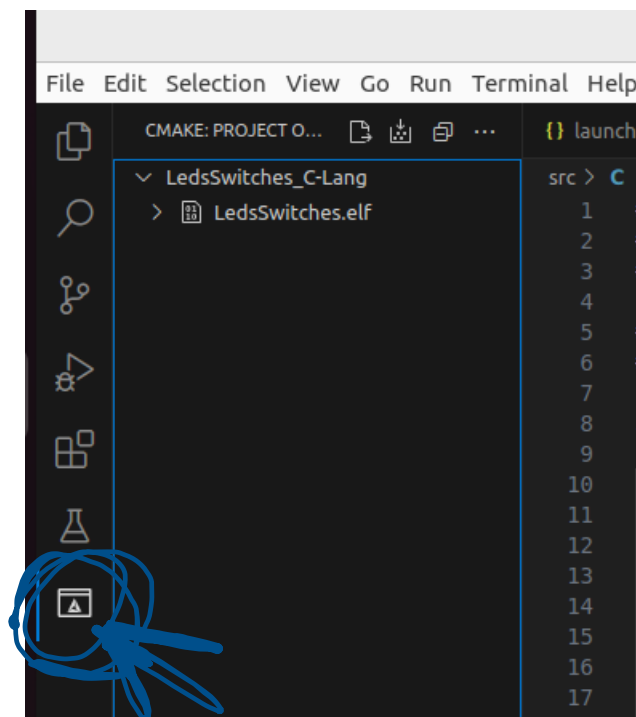
After that we will go to the debug section of the IDE and make a configuration by clicking the create launch.json button.



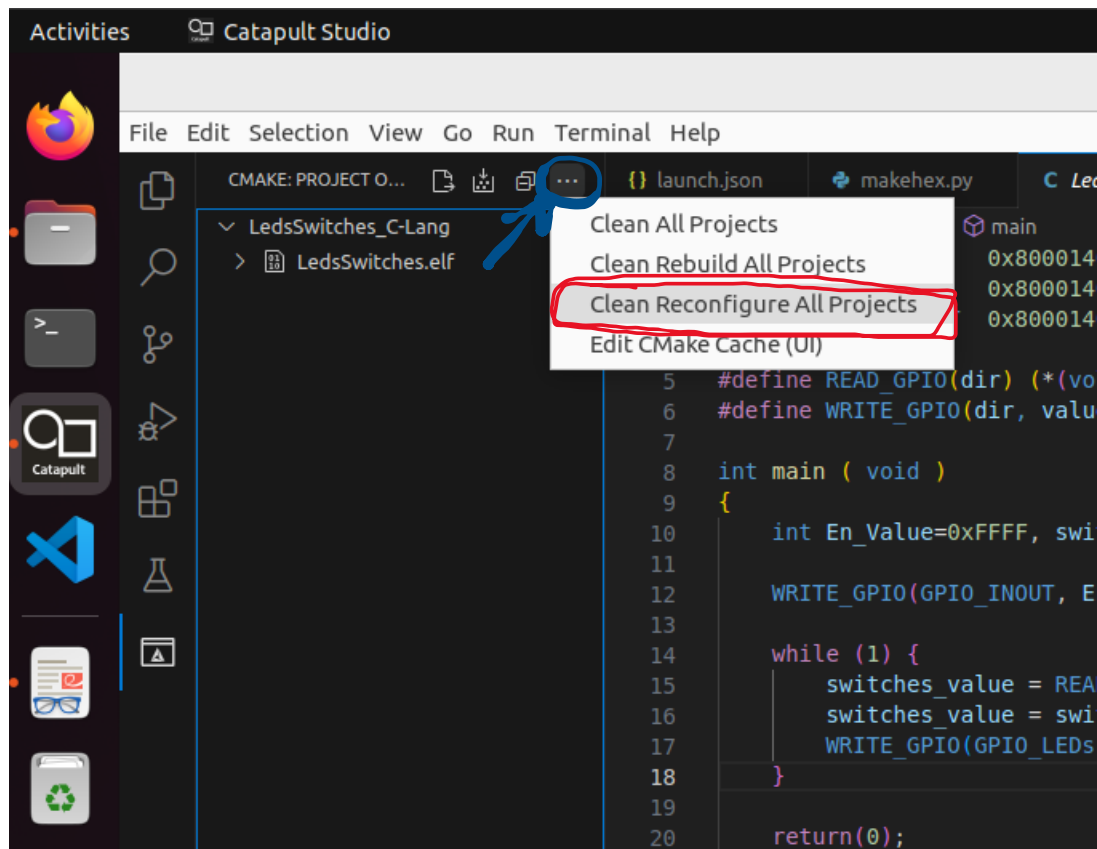
Then a panel should pop up and we choose C/C++ Embedded for the debugger.



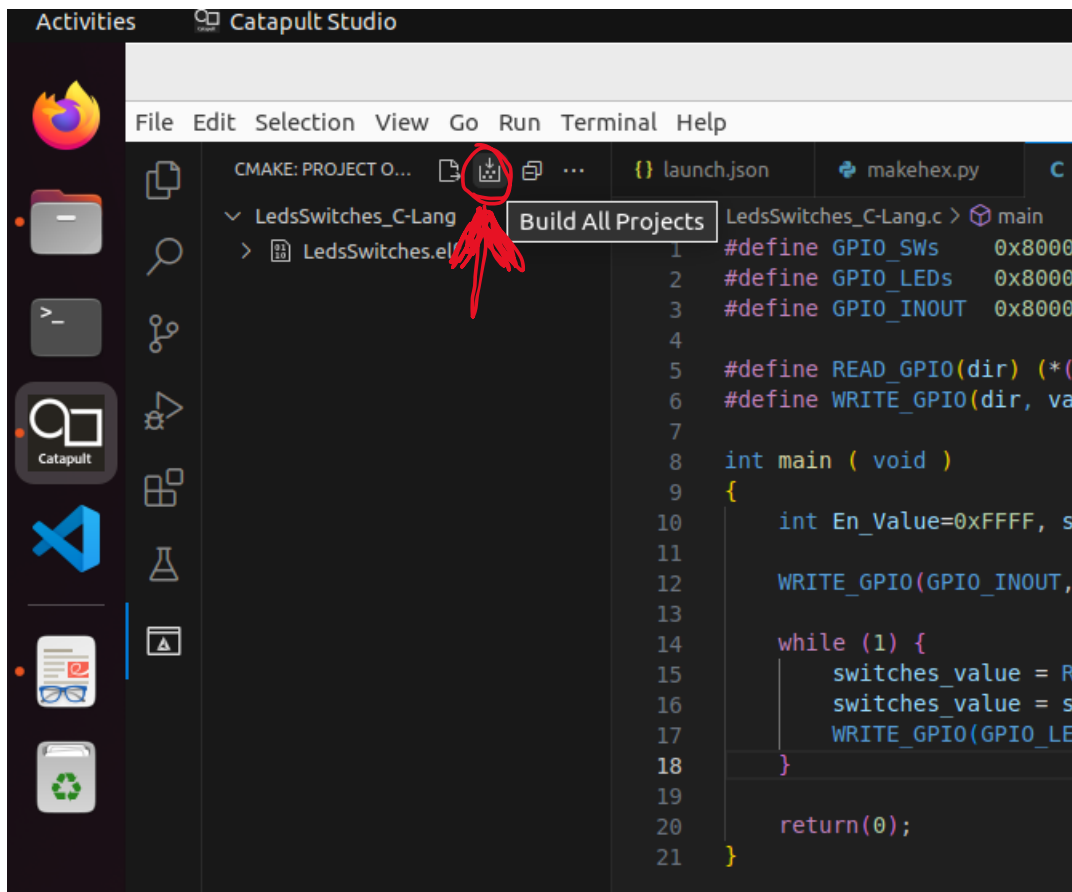
If we modified the platform, we should reconfigure all the project elf files, so before running the scripts we need go to the CMake tab.



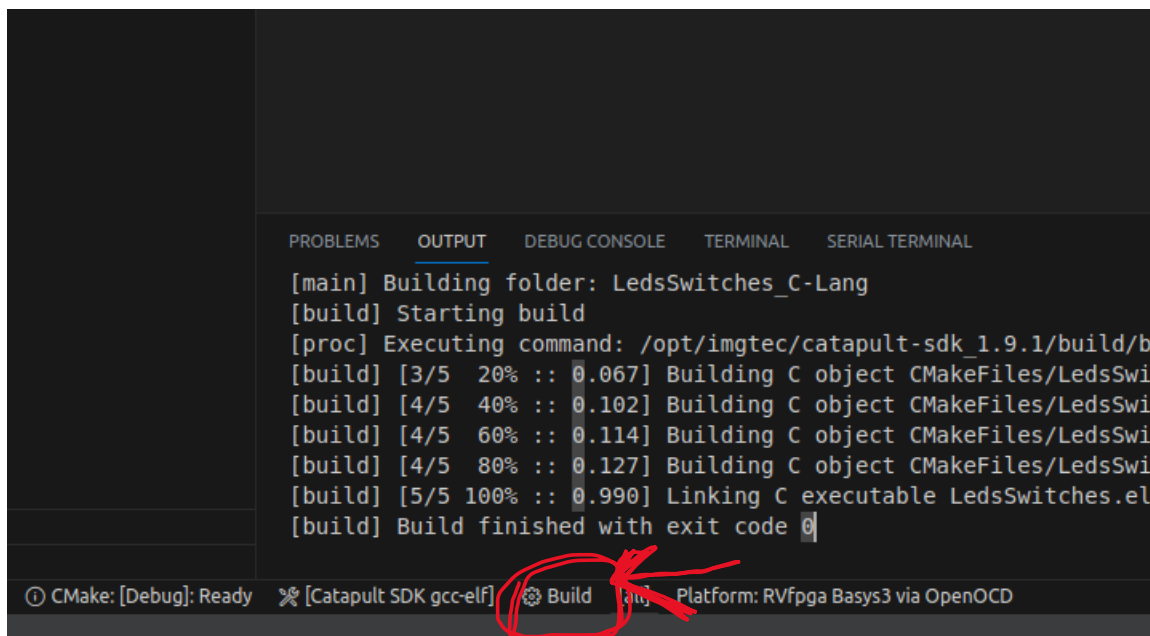
Here we'll select the three dots and run "Reconfigure all projects"



Then we'll rebuild all projects.



Now we need to build the elf by pressing the build button.



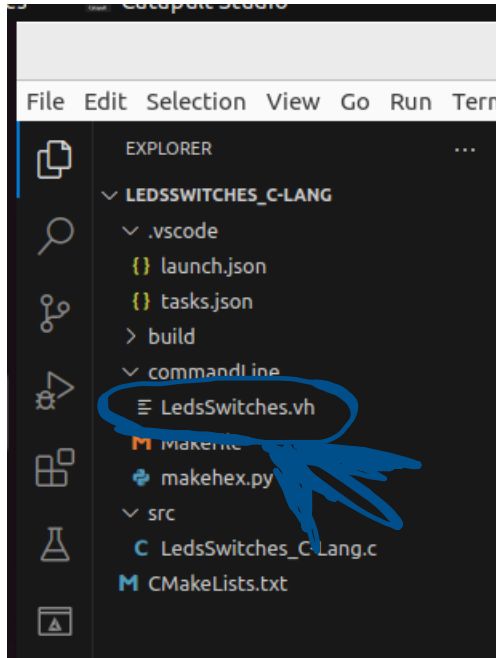
If everything worked fine, the build should go up to 100% and exit with code 0.

Now we're ready to run the code and start the simulation.

Open up a terminal in catapult sdk and change directory to the commandline folder.

Make sure you have the RiscV toolchain ready to go in catapult sdk. If you're using the provided Ubuntu VM it should be easy to do that. Just run the command `make clean` and `"export CATAPULT_SDK_TOPDIR="/opt/imgtec/catapult-sdk_1.9.1""`.

If you already have the `catapult_sdk_topdir` settled up, simply run the command `make LedsSwitches.vh`.



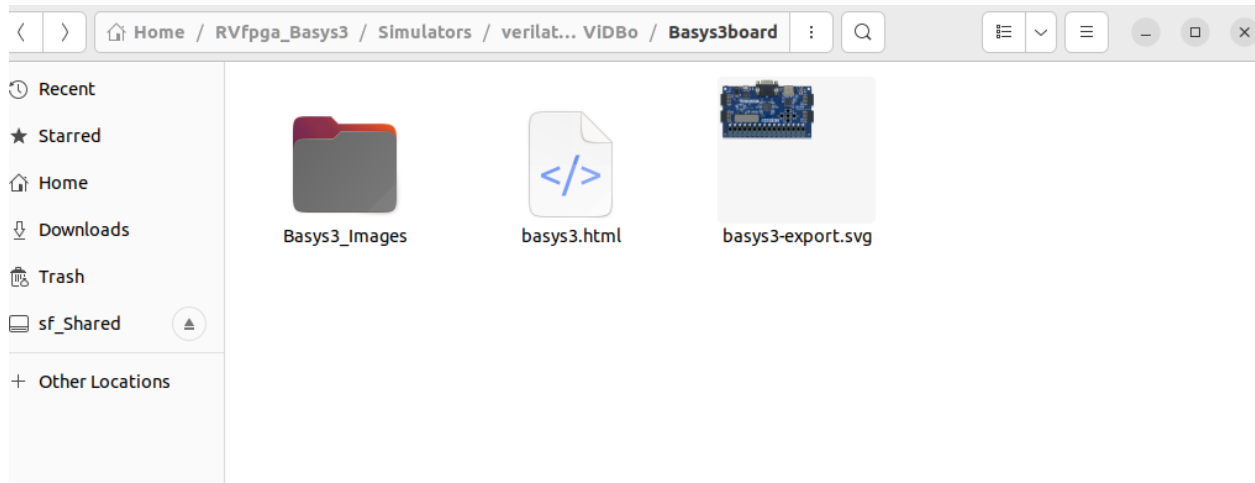
Therefore the `LedsSwitches.vh` file should be created. This file is the one that sets the parameters in the verilog simulation. Now that we have everything ready we just have to run the simulation.

Firstly we'll open up a terminal in the Simulators->verilatorSIM_ViDBo->Basys3Board (or whatever board you chose).

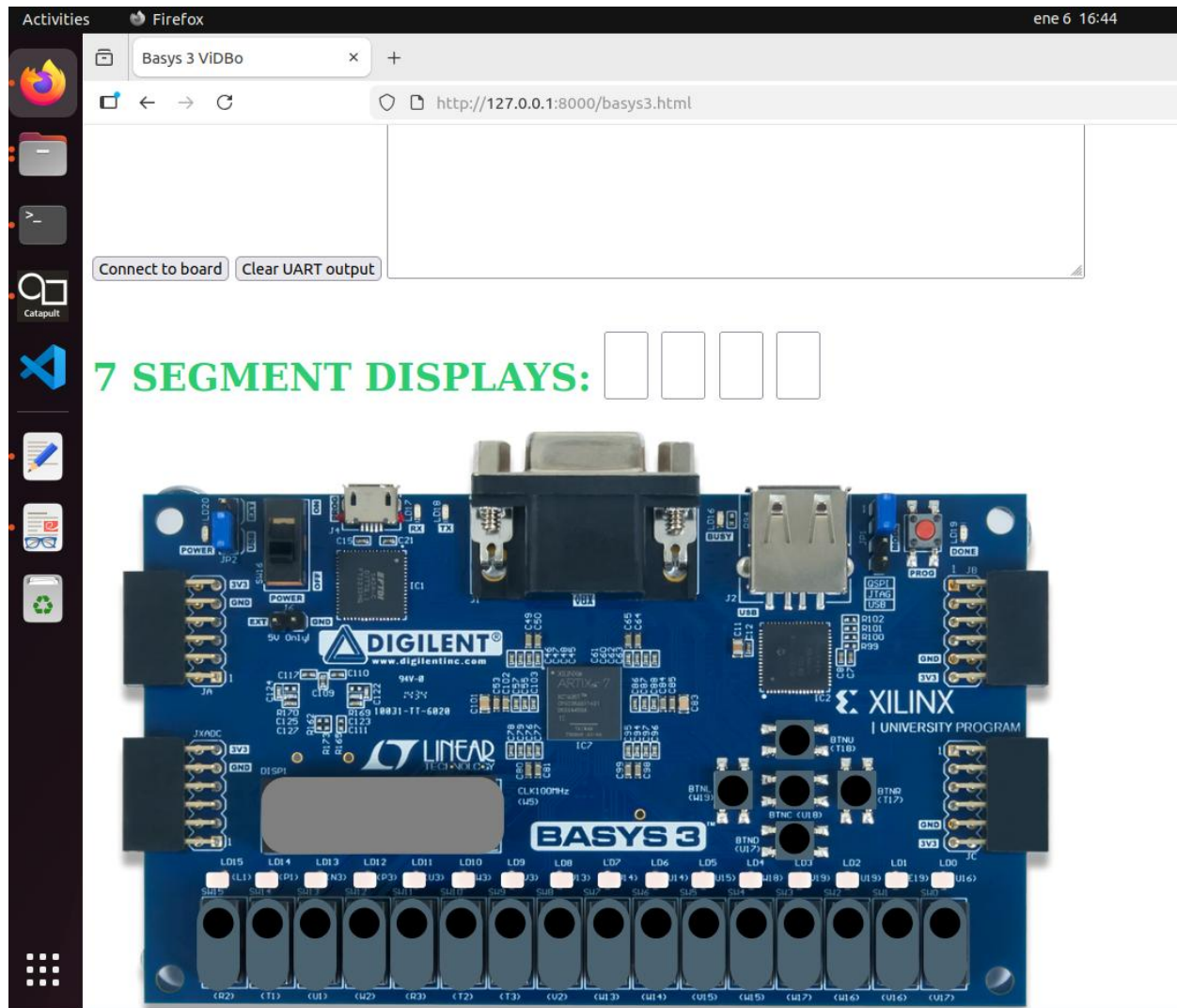
Here we'll run the command `"python3 -m http.server"` to create a local server that hosts our web based visual application.

```
Bo/Basys3board/http.server : [Errno 2] No such file or directory
rvfpga@rvfpga-VirtualBox:~/RVfpga_Basys3/Simulators/verilatorSIM_VIDBo/Basys3boa
rd$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

If everything worked fine we can open up a browser and move to that address 0.0.0.0:8000/basys3.html or how the html is named in the Simulation-> Vidbo-> [Platform]



If everything worked until now we should have a visual of the board in our browser.



Now we go back to the CatapultSDK, in the terminal from where we left and run the command

```
“../Simulators/verilatorSIM_ViDBo/Vrvfpgasim  
+ram_init_file=LedsSwitches.vh”
```

It's very important to run it exactly like in the photo, no spacing or anything else, otherwise the command will not work and the parameters will not be updated in the verilator simulation!!

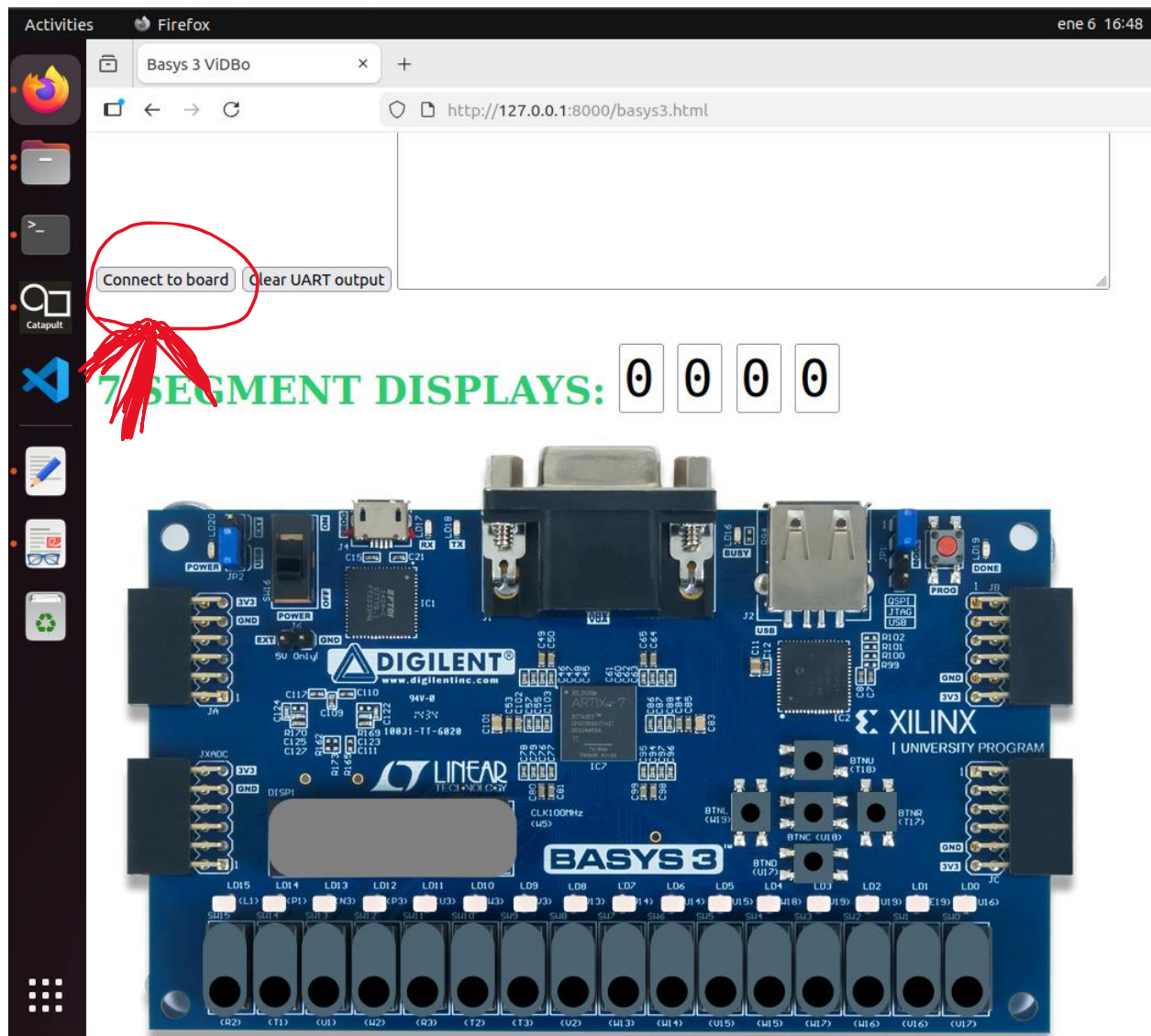
```

rvfpga@rvfpga-VirtualBox:~/RVfpga_Basys3/examples/LedsSwitches_C-Lang/commandLine$ ../../Simulators/verilatorSIM_VidBo/Vrvfpgasim +ram_init_file=LedsSwitches.vh
HELLO_TB
Loading RAM contents from LedsSwitches.vh

```

If everything worked properly, we should have in the terminal “Loading RAM contents”.

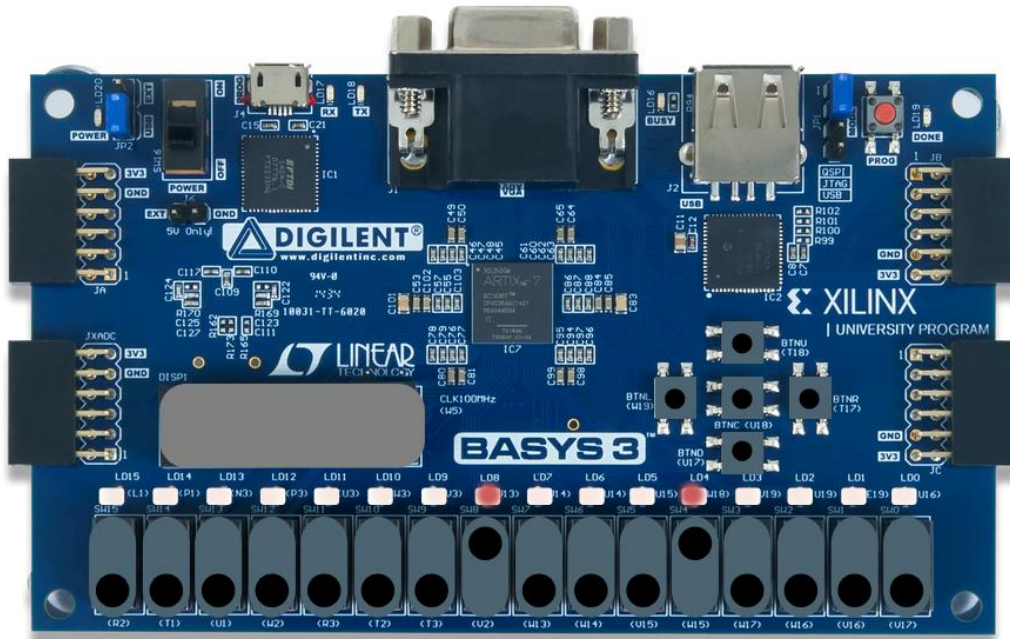
Now we can go back to the browser and connect to the board.



Disconnect Clear UART output

7 SEGMENT DISPLAYS:

0 0 0 0



And now the program should work.

Because of the precision needed to run the init files into ram, certainly we will make a mistake 80% of the time, so I made 3 shell scripts to automate the process.

```
Open  automate_run.sh
~/Own_Projects

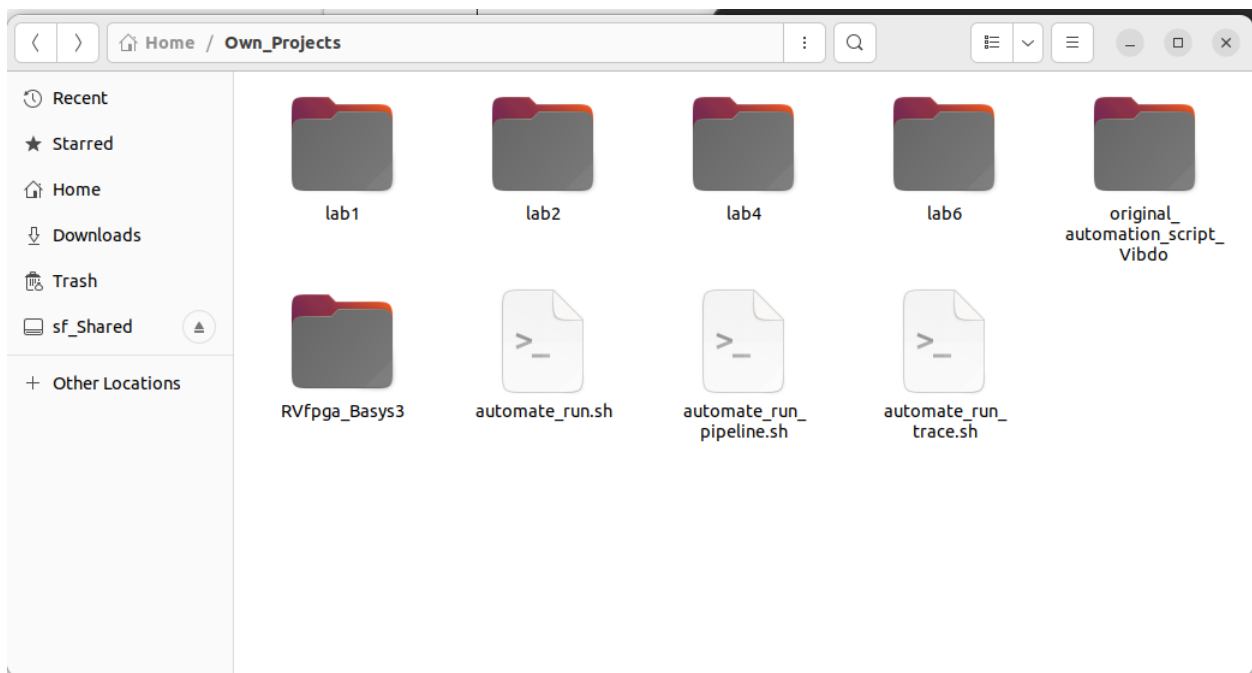
1 if [ $# -lt 1 ]; then
2     echo "Usage: $0 <raw_init_file>"
3     exit 1
4 fi
5
6 SIMULATION_PATH=~/Own_Projects/RVfpga_Basys3/Simulators/verilatorSIM_ViDBo/
7 RAW_INIT_FILE=$1
8
9 if [ ! -f "$RAW_INIT_FILE" ]; then
10     echo "Error: File '$RAW_INIT_FILE' not found!"
11     exit 1
12 fi
13
14 echo "Python server starting on port 8000"
15 python3 -m http.server --directory ~/Own_Projects/RVfpga_Basys3/Simulators/verilatorSIM_ViDBo/Basys3board/ & SERVER_PID=$!
16
17 trap "kill $SERVER_PID" EXIT
18
19 sleep 2
20
21 /home/rvfpga/Own_Projects/RVfpga_Basys3/Simulators/verilatorSIM_ViDBo/Vrvfpgasim +ram_init_file="$RAW_INIT_FILE"
```

This script only asks for the path of the .vh file and will do everything else by it's own including running the server. Keep in mind that you have to change the 3 paths I highlighted there to fit the paths in your system.

```
5
6 SIMULATION_PATH=~/Own_Projects/RVfpga_Basys3/Simulators/verilatorSIM_ViDBo/
7 RAW_INIT_FILE=$1
8
9 if [ ! -f "$RAW_INIT_FILE" ]; then
10     echo "Error: File '$RAW_INIT_FILE' not found!"
11     exit 1
12 fi
13
14 echo "Python server starting on port 8000"
15 python3 -m http.server --directory ~/Own_Projects/RVfpga_Basys3/Simulators/verilatorSIM_ViDBo/Basys3board/ &
16
17 trap "kill $SERVER_PID" EXIT
18
19 sleep 2
20
21 /home/rvfpga/Own_Projects/RVfpga_Basys3/Simulators/verilatorSIM_ViDBo/Vrvfpgasim -ram_init_file="$RAW_INIT_F
```

The other 2 scripts have 2 of the three paths, since the python server isn't needed for them.

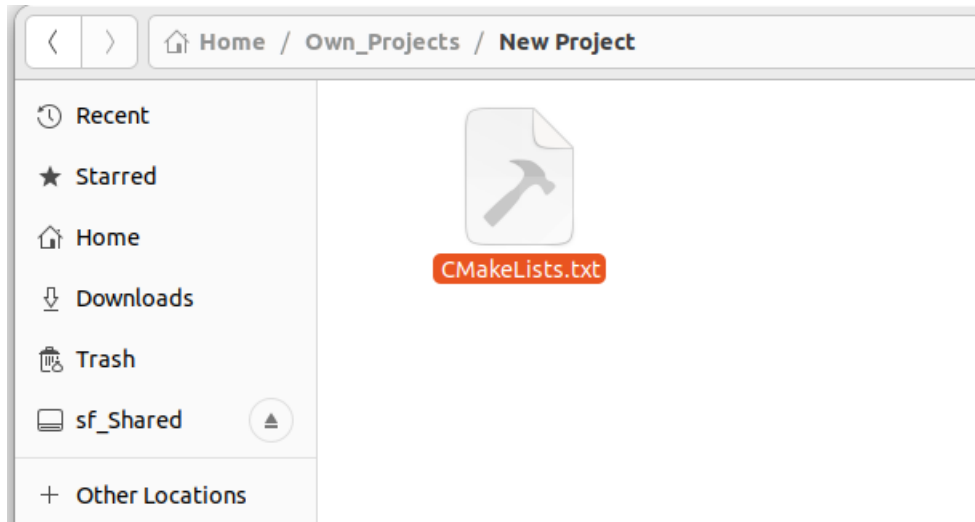
Now that we know how to build examples, we'll learn how to
Write a project from scratch



This is my structure yet. I recommend using the same kind of structure since it would be easier to use the `automate_run` with the relative path to the init files.

So, first of all we create a new folder in which we'll work.

Here, the first thing we need to do is to create a `CMakeLists.txt` that expresses the tools we'll use and where will they go.

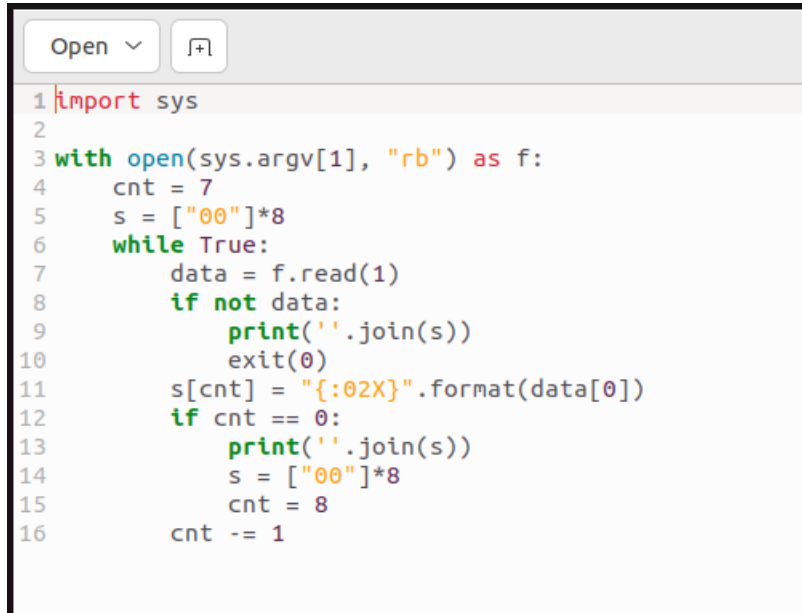


Here we have to follow the next structure:

```
1 cmake_minimum_required(VERSION 3.17)
2
3 project(Example C)
4
5 set(SOURCE
6
7     ${CMAKE_CURRENT_SOURCE_DIR}/src/Example.c
8
9 )
10
11 set(TARGET_NAME Example.elf)
12
13 add_executable(${TARGET_NAME} ${SOURCE})
14
15 include(${CMAKE_CURRENT_SOURCE_DIR}/../RVfpga_Basys3/common/Common.cmake)
16
17
```

We can replace “Example” with any name we like, but we have to keep in mind that the files we generate with make in the commandline folder will take that name, in this case “Example.vh” or “Example.dis” for the disassembly.

Now we create a src folder in which we write our “Example.c” file. Then we need a commandline folder in which we add a makehex.py using the following structure:



```
1 import sys
2
3 with open(sys.argv[1], "rb") as f:
4     cnt = 7
5     s = ["00"]*8
6     while True:
7         data = f.read(1)
8         if not data:
9             print(''.join(s))
10            exit(0)
11            s[cnt] = "{:02X}".format(data[0])
12            if cnt == 0:
13                print(''.join(s))
14                s = ["00"]*8
15                cnt = 8
16            cnt -= 1
```

And a Makefile with the following structure:



```
1 TOOLCHAIN_PREFIX=${CATAPULT_SDK_TOPDIR}/bin/riscv64-img-elf-
2 RESET_VECTOR ?= 0
3 FLASH_ADDR ?= 0x0
4 DIR = ../build
5
6 %.vh: %.bin
7     python3 makehex.py $< > $@
8 %.bin: $(DIR)/%.elf
9     $(TOOLCHAIN_PREFIX)objcopy -O binary $< $@
10 %.dis: $(DIR)/%.elf
11     $(TOOLCHAIN_PREFIX)objdump -d $< > $@
12 clean:
13     rm -f *.bin *.vh *.dis *.vcd
```

If you did everything right until this point, we just open the project in CatapultSDK and do the steps that we did previously a few pages above.

After generating the .vh file we go back to the main project folder and run the automate_run.sh script to the file generated and everything should work perfectly

